

Oui, et c'est même une très bonne idée de faire une pause maintenant. Tu as utilisé plusieurs technologies, mais il faut comprendre **qui fait quoi et pourquoi**, sinon on risque d'apprendre les commandes sans vraiment comprendre l'architecture.

Je vais te l'expliquer comme une **carte mentale de ton projet Delivery Planner**, avec des mots simples.

1. D'abord : les grandes catégories

Dans ton projet, tu as plusieurs choses qui ne sont pas de même nature :

Élément	C'est quoi ?	Rôle
Java	Langage de programmation	Écrire ton backend
Spring Boot	Framework Java	Construire facilement ton application web/backend
REST	Style d'architecture/API	Permettre au frontend de communiquer avec le backend
PostgreSQL	Système de gestion de base de données	Stocker les données
SQL	Langage	Parler directement à une base de données
JPA	Spécification Java	Faire le lien entre objets Java et tables SQL
Hibernate	Implémentation de JPA	Effectuer concrètement le travail ORM
Maven	Outil de gestion de projet	Gérer les dépendances et construire le projet
MVC	Architecture/conception	Organiser le code en différentes responsabilités
Eclipse	IDE	Écrire, lancer et déboguer ton code

Le point important :

Java, PostgreSQL, Spring Boot, JPA, Hibernate, Maven, MVC ne sont pas tous des "langages" ou des "frameworks".

Ils ont des rôles différents.

2. Java : le langage

Java est un langage de programmation.

Tu écris par exemple :

Java

```
public class Delivery {  
  
    private Long id;  
    private String client;  
    private String address;  
}
```

Java permet de créer :

- des classes
- des objets
- des méthodes
- des variables
- des conditions
- des boucles
- etc.

Dans ton projet :

Java

↓

code de ton backend

Par exemple :

Java

```
public Delivery addDelivery(Delivery delivery) {  
    return deliveryRepository.save(delivery);  
}
```

C'est du **Java**.

3. Spring Boot : le framework

Spring Boot n'est pas un langage.

C'est un **framework Java**.

Un framework fournit une structure et des outils pour développer plus facilement une application.

Sans Spring Boot, construire un serveur HTTP en Java demanderait beaucoup plus de configuration.

Avec Spring Boot, tu peux écrire :

Java

```
@RestController
public class DeliveryController {

    @GetMapping("/api/deliveries")
    public List<Delivery> getDeliveries() {
        ...
    }
}
```

Et Spring Boot comprend :

"Cette classe reçoit des requêtes HTTP."

Pourquoi Spring Boot ?

Pour ton projet :

```
Java
+
Spring Boot
↓
Backend web
```

Spring Boot s'occupe notamment de :

- démarrer le serveur
- gérer les requêtes HTTP
- connecter différents composants
- gérer la configuration
- intégrer JPA/Hibernate
- etc.

Dans ta console, tu as vu :

Tomcat started on port 8080

Spring Boot a démarré un serveur web **Tomcat** pour ton application.

4. REST : comment les applications communiquent

REST n'est **pas un langage**.

C'est une manière de concevoir une API utilisant généralement HTTP.

Dans ton projet, tu as :

```
GET /api/deliveries
```

et :

```
POST /api/deliveries
```

Ce sont des endpoints de ton API REST.

Les principales méthodes

Méthode	Signification
GET	récupérer
POST	créer
PUT	modifier
DELETE	supprimer

Par exemple :

```
GET /api/deliveries
```

signifie :

"Donne-moi les livraisons."

Alors que :

```
POST /api/deliveries
```

signifie :

"Voici une nouvelle livraison, ajoute-la."

5. PostgreSQL : oui, c'est une base de données ?

Presque : plus précisément, **PostgreSQL** est un **système de gestion de base de données relationnelle** (SGBDR).

La **base de données** elle-même est :

```
delivery_planner
```

PostgreSQL est le logiciel qui permet de la gérer.

Tu peux imaginer :

```
PostgreSQL
├──
└── delivery_planner
    ├── delivery
    ├── driver
    └── ...
```

Une table

Une table pourrait ressembler à :

	id	client	address
	1	Client A	Brest
	2	Client B	Plouzané
	3	Client C	Quimper

Donc :

PostgreSQL = logiciel/SGBD

delivery_planner = ta base de données

delivery = une table

6. SQL : le langage des bases de données

SQL est un langage, contrairement à PostgreSQL.

Par exemple :

SQL

```
SELECT * FROM delivery;
```

signifie :

recupérer toutes les lignes de la table `delivery`.

Pour ajouter :

SQL

```
INSERT INTO delivery (client, address)
VALUES ('Client A', 'Brest');
```

Donc :

SQL = langage
PostgreSQL = logiciel qui comprend SQL

C'est un peu comme :

Java = langage
JVM = environnement qui permet d'exécuter Java

7. JPA : c'est quoi ?

C'est probablement le concept le plus important à comprendre maintenant.

JPA = Java Persistence API.

Mais attention :

JPA n'est pas une base de données.

JPA n'est pas un langage.

JPA n'est pas directement Hibernate.

JPA est une **spécification** qui définit comment des objets Java peuvent être persistés dans une base de données relationnelle.

Le mot important est **persistance**.

Persistance = conserver les données

Avant :

Java

```
List<Delivery> deliveries = new ArrayList<>();
```

Les données étaient en mémoire.

Si tu arrêtais ton programme :

Programme arrêté

↓

données perdues

Avec PostgreSQL :

Java

↓

JPA/Hibernate

↓

PostgreSQL

↓

données conservées

Même si tu arrêtes Spring Boot, les données restent dans PostgreSQL.

8. Hibernate : alors c'est quoi ?

Hibernate est un ORM.

ORM signifie :

Object-Relational Mapping

En français :

correspondance entre objets et données relationnelles.

C'est justement le lien entre :

Objets Java

et :

Tables SQL

Par exemple ton objet Java :

Java

Delivery

peut correspondre à une table :

delivery

Et :

Java

```
private Long id;  
private String client;  
private String address;
```

correspondent aux colonnes :

id
client
address

Schématiquement :

JAVA

Delivery

id

client

address

SQL

delivery

id

client

address

←→

←→

←→

Hibernate s'occupe de faire cette correspondance.

9. JPA vs Hibernate

C'est une distinction importante pour les entretiens.

Tu peux retenir :

JPA = les règles / l'interface

Hibernate = l'outil qui applique ces règles

Une analogie :

```
JPA      = contrat / spécification
Hibernate = implémentation du contrat
```

Dans ton code, tu écris :

Java

```
import jakarta.persistence.Entity;
import jakarta.persistence.Id;
```

Puis :

Java

```
@Entity
public class Delivery {
```

Ces annotations viennent de l'écosystème **JPA**.

Mais derrière, c'est **Hibernate** qui fait concrètement le travail avec PostgreSQL.

Et tu as vu dans ta console :

```
Hibernate ORM core version 7.4.5.Final
```

Donc Hibernate est bien présent dans ton application.

10. Pourquoi `@Entity` ?

Quand tu écris :

Java

```
@Entity
```

tu dis essentiellement à JPA/Hibernate :

"Cette classe Java doit être considérée comme une entité persistante."

Donc Hibernate peut créer une correspondance avec une table.

11. Et `@Id` ?

Tu as :

Java

```
@Id
private Long id;
```

Cela signifie :

`id` est la clé primaire de cette entité.

Dans la table :

delivery

on aura quelque chose comme :

	id	client	address
	1	Client A	Brest
	2	Client B	Quimper

Chaque ligne possède un identifiant unique.

12. `@GeneratedValue`

Tu as également :

Java

```
@GeneratedValue(strategy = GenerationType.IDENTITY)
```

Cela signifie que l'identifiant est généré automatiquement.

Donc tu peux envoyer :

JSON

```
{  
  "client": "Client C",  
  "address": "Quimper"  
}
```

Tu ne fournis pas :

JSON

```
"id": 3
```

PostgreSQL génère l'identifiant.

13. Repository : pourquoi ?

Tu as créé :

Java

```
public interface DeliveryRepository  
    extends JpaRepository<Delivery, Long> {  
}
```

Le repository est une couche qui permet au code Java d'accéder aux données.

Par exemple :

Java

```
deliveryRepository.findAll();
```

→ récupérer les livraisons.

Java

```
deliveryRepository.save(delivery);
```

→ enregistrer une livraison.

Java

```
deliveryRepository.findById(id);
```

→ chercher une livraison.

Java

```
deliveryRepository.deleteById(id);
```

→ supprimer une livraison.

Tu n'as même pas besoin d'écrire toi-même le SQL pour ces opérations.

14. Spring Data JPA

Tu as également rencontré quelque chose appelé **Spring Data JPA**.

C'est encore un niveau différent :

```
Spring Boot
  ↓
Spring Data JPA
  ↓
JPA
  ↓
Hibernate
  ↓
PostgreSQL
```

Spring Data JPA facilite l'utilisation de JPA dans Spring.

C'est pour cela que tu peux simplement écrire :

Java

```
extends JpaRepository<Delivery, Long>
```

et obtenir beaucoup de fonctionnalités automatiquement.

15. MVC : et maintenant MVC

MVC signifie :

Model – View – Controller

C'est un **pattern architectural** permettant d'organiser une application.

Model

Le **Model** représente les données et parfois la logique liée aux données.

Dans ton projet :

```
Delivery
```

est ton modèle de données.

Java

```
public class Delivery {  
    private Long id;  
    private String client;  
    private String address;  
}
```

Controller

Le Controller reçoit les requêtes.

Ton :

Java

```
DeliveryController
```

contient :

Java

```
@GetMapping("/api/deliveries")
```

et :

Java

```
@PostMapping("/api/deliveries")
```

Donc :

```
Client
```

```
↓ HTTP
```

```
Controller
```

View

Dans un MVC classique, la View correspond à ce que l'utilisateur voit.

Par exemple :

```
HTML
CSS
JavaScript
```

Dans ton projet, tu vas avoir un **frontend React**.

Donc ton architecture globale sera plutôt :

```
React
  ↓ HTTP/JSON
Spring Boot
  ↓
Controller
  ↓
Service
  ↓
Repository
  ↓
JPA/Hibernate
  ↓
PostgreSQL
```

C'est pour ça que ton projet n'est pas un MVC "pur" avec une View générée directement par Spring : ton frontend React est séparé du backend.

16. Et pourquoi avons-nous Service ?

Tu as :

```
Controller
  ↓
Service
  ↓
Repository
```

Le **Service** contient la logique métier.

Par exemple, plus tard tu pourrais avoir :

Java

```
public Delivery assignDelivery(...) {  
    // vérifier le chauffeur  
    // vérifier la distance  
    // choisir un chauffeur  
    // affecter la livraison  
}
```

Le Controller ne devrait pas contenir toute cette logique.

Donc :

Controller

"Quelqu'un demande quelque chose."

Service

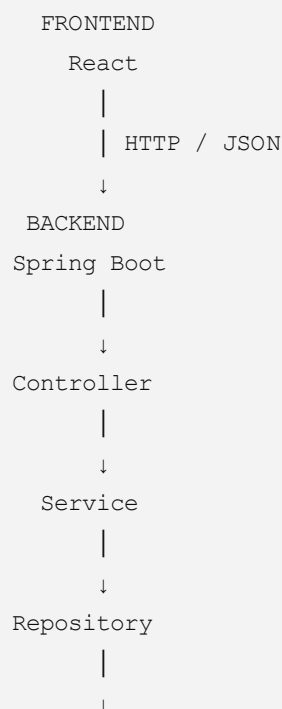
"Voici comment l'application traite cette demande."

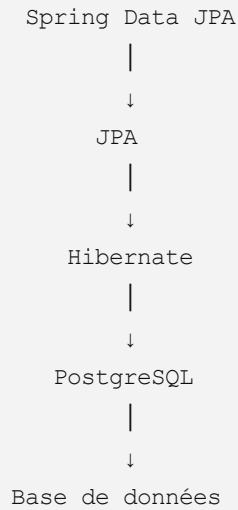
Repository

"Voici comment je récupère ou sauvegarde les données."

17. Ton architecture actuelle complète

Voilà probablement le schéma le plus important à retenir :





Et autour :

Java → langage
Maven → gestion du projet/dépendances
Eclipse → environnement de développement

18. La distinction à retenir absolument

Si on te demande en entretien :

"Quelles technologies utilisez-vous ?"

Tu peux dire :

« J'utilise Java avec Spring Boot pour développer le backend REST, Spring Data JPA et Hibernate pour la persistance des données, et PostgreSQL comme système de gestion de base de données relationnelle. »

Et si on te demande :

"JPA et Hibernate, c'est la même chose ?"

Tu réponds :

« Non. JPA est une spécification Java pour la persistance des données, tandis qu'Hibernate est une implémentation ORM qui permet de mettre en œuvre cette spécification. »

Et :

"PostgreSQL est un langage ?"

« Non, PostgreSQL est un système de gestion de base de données relationnelle. Il utilise notamment SQL comme langage de requête. »

Ta fiche mentale

Tu peux retenir seulement ça pour l'instant :

JAVA

= langage

SPRING BOOT

= framework Java pour créer le backend

REST

= manière de concevoir l'API

POSTGRESQL

= système de gestion de base de données

SQL

= langage des bases de données

JPA

= spécification Java pour la persistance

HIBERNATE

= ORM / implémentation de JPA

SPRING DATA JPA

= facilite l'utilisation de JPA dans Spring

REPOSITORY

= accès aux données

SERVICE

= logique métier

CONTROLLER

= reçoit les requêtes HTTP

MVC

= manière d'organiser les responsabilités

Le plus important : ne cherche pas à mémoriser toutes les définitions mot pour mot. Pour ton projet, il faut surtout comprendre le **chemin d'une donnée** :

POST /api/deliveries → Controller → Service → Repository → JPA/Hibernate → PostgreSQL

C'est exactement ce que tu viens de construire.



 Coursera

Ad

Coursera Data Analytics

Learn SQL, Python and Tableau